

SZABIST
UNIVERSITY
Islamabad Campus

Final Year Project Document

Multi-Agent Market Research Platform

Chapter 3 — Software Requirements Specification

Aroon Kumar (2312466)
Mudasir Ali (2312465)
Abdullah Zafar Khan (2312467)

Supervised by: Mr. Zubair Ahmed

Department of Department of Computer Science
Faculty of Computing and Engineering Sciences
SZABIST University
SZABIST (Islamabad Campus), Pakistan

Fall 2026

Revision History

Compiled By	Checked By	Date	Description	Version
Aroon Kumar	Mr. Zubair Ahmed	15 Sept 2026	Section 1	1.0
Mudasir Ali	Mr. Zubair Ahmed	15 Sept 2026	Section 2	1.1
Abdullah Zafar Khan	Mr. Zubair Ahmed	15 Sept 2026	Section 3	1.2
Aroon Kumar	Mr. Zubair Ahmed	16 Sept 2026	Section 4	1.3
Mudasir Ali	Mr. Zubair Ahmed	16 Sept 2026	Section 5	1.4
Abdullah Zafar Khan	Mr. Zubair Ahmed	16 Sept 2026	Section 6	1.5
Aroon Kumar	Mr. Zubair Ahmed	17 Sept 2026	Section 7	1.6
Mudasir Ali	Mr. Zubair Ahmed	17 Sept 2026	Section 8	1.7
Abdullah Zafar Khan	Mr. Zubair Ahmed	17 Sept 2026	Section 9	1.8
Aroon Kumar	Mr. Zubair Ahmed	18 Sept 2026	Section 10	1.9
Mudasir Ali	Mr. Zubair Ahmed	18 Sept 2026	Section 11	2.0
Abdullah Zafar Khan	Mr. Zubair Ahmed	18 Sept 2026	Section 12, 13	2.1

Chapter 3

Software Requirements Specification

A Software Requirements Specification (SRS) defines the expected software behaviour, system boundaries and quality constraints used to assess an implementation. It provides developers, reviewers and intended users with a common basis for implementation and acceptance testing. For the proposed system, the chapter specifies company access, questionnaires, campaigns, Android field collection, offline synchronization, media handling and AI-assisted research. The requirements describe the behaviour to be delivered and the conditions under which it is assessed. Detailed screen designs and database schemas belong to the design stage [1].

Section 3.1 introduces the product scope and intended readership of the specification. Section 3.2 defines ten functional requirements covering the company and respondent workflows. Section 3.3 states quality constraints for response time, company data protection, offline records and Android resource use. Section 3.4 explains how technical consultation, document analysis and frontend prototyping inform the requirements. Section 3.5 presents the planned Scrum sprints and their increments.

3.1 Introduction

The proposed system connects a company web portal with an Android application for on-ground market research. Field staff collect questionnaire responses and supporting photographs or audio at assigned locations. The application saves records offline and sends pending data to the server when connectivity is available. Server-side agents collect information from approved public web pages and supplied documents, extract relevant details, check the evidence and prepare research summaries. LangGraph coordinates these tasks within defined permissions, while uncertain records remain subject to human review. Outdoor advertising audits are one supported field-research use case.

Document Scope

The document specifies the functional requirements and quality constraints of the product. The React portal supports company administration, questionnaires, campaigns and reports. The Android application uses Kotlin and local SQLite storage for field collection. FastAPI services connect the clients with PostgreSQL records and the LangGraph research workflow. GPS coordinates support assigned-site attendance checks and campaign or location comparisons. The scope excludes physical billboard fabrication, hardware maintenance and telecom infrastructure management. A separate Coding Agent

supports implementation through assistance reviewed by the development team.

Audience

The specification gives the development team implementation requirements, product owners and technical reviewers acceptance criteria, and operational users a description of the supported tasks. The same requirements guide implementation checks, integration testing and product acceptance.

- **Development Team:** The team uses the requirements to develop the React portal, Kotlin Android client, FastAPI services, database records and coordinated AI tasks.
- **Product Owners and Technical Reviewers:** Reviewers compare the implementation and test results with the product requirements and defined scope.
- **Testing Team:** The team uses the requirements to check user actions, company access, offline saving, synchronization and reported outputs.
- **Company Research Staff:** Company administrators, research managers, field staff, analysts and reviewers check whether the described tasks match their assigned responsibilities.

3.2 Functional Requirements

Functional requirements describe the operations that the proposed system performs, the information it accepts and the outputs it produces. They provide a basis for implementation and functional testing [1]. Table 3.1 defines ten product workflows. Related capabilities share records and supporting operations, including account access, media links and collection identifiers.

Administrator means the administrator of an individual company workspace. Researcher means an authorised research manager or analyst, Surveyor means a field staff member, Reviewer means an assigned evidence reviewer, and Respondent means a survey participant. Access to each workflow is restricted to the relevant company and assigned role. Each requirement identifies a primary use case; related role actions and exception paths form part of that workflow.

Table 3.1 links each functional requirement to its corresponding use case, allowing reviewers to trace system behavior back to the needs of its users.

3.3 Non-Functional Requirements

Non-functional requirements state the quality levels and operating constraints that the proposed system must satisfy. They describe how well the specified functions operate,

Table 3.1: Functional requirements of the proposed system

Requirement ID	Description
FR-001	Administrator can manage access.
FR-002	Researcher can manage surveys.
FR-003	Researcher can manage assignments.
FR-004	Surveyor can record evidence.
FR-005	Surveyor can sync submissions.
FR-006	Reviewer can validate submissions.
FR-007	Researcher can compare competitors.
FR-008	Researcher can coordinate workflows.
FR-009	Researcher can review performance.
FR-010	Researcher can analyze campaigns.

including security, reliability, usability, compatibility and performance [1]. The following requirements apply to the web portal, Android client and backend. The numerical values are acceptance targets for evaluation; they do not represent measured results.

Software Quality Attributes

- **Security:** Company records shall be accessible only to authorised users of that company. The application shall use HTTPS for data transferred to the backend. Account protection shall rely on the standard safeguards of the selected, maintained authentication component. Company administrators shall receive no access to other company workspaces.
- **Reliability:** Saved Android responses shall remain available after an application restart or an interrupted connection. Repeated synchronization attempts shall retain collection identifiers and shall not create duplicate accepted submissions. Pending records shall remain on the device until the server confirms successful receipt.
- **Usability:** Questionnaires shall provide readable English and Urdu content where configured, clear labels and specific validation messages. Submission states shall distinguish saved, pending, synchronized and failed records. The survey interface shall explain the purpose of collection and intended data use before participation. Deletion of saved research records shall require confirmation.
- **Maintainability:** The implementation shall separate the web client, Android client, backend and agent code through documented interfaces. The team shall maintain version-controlled source code and reproducible setup instructions.
- **Compatibility:** The web portal and Android application shall exchange data through the same documented API fields and formats. Question identifiers and

collection identifiers shall retain their meaning when records move between the device and server.

Performance Requirements

Routine web requests shall complete within three seconds for at least 95% of requests in an evaluation with ten concurrent users and 5,000 stored survey responses. The target excludes media transfers and external AI processing. The Android application shall save a completed response locally within two seconds. On a stable connection of at least 10 Mbps, the app shall upload twenty queued responses without photographs or audio within thirty seconds. It checks the transfer of responses saved while offline; media transfer time depends on file size.

Safety Requirements

The Android application shall limit GPS capture, media recording and preliminary AI processing to the collection or verification tasks that require them. It shall avoid continuous background AI processing and repeated network retries when connectivity is unavailable. These constraints limit unnecessary battery use and processing heat [2]. The application shall present task and synchronization updates inside the app without audible or vibrating alerts. Field staff shall enter data while stationary and review updates after travelling.

Other Non-Functional Requirements

The web portal shall operate in the Chrome and Edge versions used for evaluation. Deployment instructions shall identify the tested Android versions and required API configuration. The Android application shall access the camera, microphone and location only when the required device permissions are granted. Local and server records shall remain subject to the company access restrictions stated in the specification.

3.4 Requirements Gathering Techniques Used

Requirements gathering identifies the needs, constraints and expectations that guide implementation [1]. The product requirements baseline combines technical consultation, analysis of relevant documents and review of the questionnaire prototype. These activities clarify product boundaries, collection workflows and expected behaviour. The documented consultation concerns technical review and scope clarification. Direct validation with external company representatives and field staff remains pending.

Technical consultation

Technical consultation gathers requirements through discussion of intended behaviour, implementation constraints and delivery boundaries. The reviewer and development team

clarify needs and examine how the proposed functions fit together. The team records the resulting decisions in the product requirements baseline and distinguishes technical feedback from validation by operational users [1].

The development team used technical review feedback to clarify company access, website questionnaires, offline Android collection and linked media records. The product requirements baseline records these capabilities and the boundaries of collection, verification, agent coordination and reporting. The review also clarified company responsibilities and access restrictions.

The consultation clarified product boundaries, responsibilities and implementation priorities. The documented feedback supports the technical baseline. Confirmation of the complete workflow by operational company users and field staff remains pending and is not presented as a completed consultation outcome.

Document analysis

Document analysis examines existing specifications, manuals, reports and other relevant records to identify requirements, rules and constraints. The analyst records what each source supports and identifies gaps that need clarification. The technique provides an evidence base for requirements before the team commits to implementation [3].

The analysis used the product feature baseline, the existing-system review in Chapter 2, the SWEBOK requirements chapter and official ODK Collect documentation. The feature baseline defined the delivery boundary. The ODK documentation provided a concrete example of offline form filling, location capture and media collection [4]. The review connected these documented capabilities with the corresponding workflows and separated user actions from quality constraints.

The analysis connected the documented product capabilities with company access, questionnaires, field collection, evidence review and reporting. It clarified the use of location records for attendance checks and campaign comparisons.

Prototyping

Prototyping uses an early representation of the interface or workflow to clarify requirements. Screens and working interactions make assumptions visible and help reviewers identify missing steps or confusing behaviour. The development team uses the observations to refine the requirements before building the complete implementation [1].

The team developed a React questionnaire frontend with questionnaire management and a respondent view. The prototype supported review of question design, response validation, survey access and questionnaire deletion. The work clarified interface behaviour before backend integration. It did not establish working database persistence or offline operation in the Android application.

The prototype provided a visible questionnaire workflow and clarified validation messages, respondent access and management actions. Its browser storage supported interface review. Database storage, mobile offline saving and synchronization remain implementation and evaluation tasks rather than demonstrated prototype results.

3.5 Time Frame

The planned implementation schedule contains four two-week sprints, matching the Scrum process model in Chapter 2. Each sprint produces an increment for review. The team begins with core records and questionnaires, then develops Android field collection, AI workflows and reporting. Testing and documentation accompany implementation. Sprint reviews guide adjustments to remaining tasks within the specified product scope [5].

Table 3.2 divides implementation into four consecutive sprints and shows the planned duration of each stage.

Table 3.2: Planned implementation schedule of the proposed system

Phase	**Duration**
Sprint 1: Core architecture, questionnaires and data ingestion	2 weeks (weeks 1–2)
Sprint 2: Android collection and field verification	2 weeks (weeks 3–4)
Sprint 3: Multi-agent intelligence and media verification	2 weeks (weeks 5–6)
Sprint 4: Analytics, system evaluation and deployment preparation	2 weeks (weeks 7–8)
Sprint 5: Pilot field trials, user acceptance and final deployment	2 weeks (weeks 9–10)

References

[1] "Chapter 1: Software Requirements," SWEBOK Wiki. [Online]. Available: https://swebokwiki.org/Chapter_1:_Software_Requirements. [Accessed: Oct. 26, 2026].

[2] Android Developers, "Optimize battery use for task scheduling APIs." [Online]. Available: https://developer.android.com/develop/background-work/background-tasks/optimize-battery. [Accessed: Oct. 26, 2026].

[3] International Institute of Business Analysis, "Document Analysis - Insight." [Online]. Available: https://www.iiba.org/contentassets/6baff5775d1a41f99d77385106790b0a/document-analysis–insight.pdf. [Accessed: Oct. 26, 2026].

[4] ODK, "ODK Collect," ODK Docs. [Online]. Available: https://docs.getodk.org/collect-intro/. [Accessed: Oct. 26, 2026].

[5] K. Schwaber and J. Sutherland, "The Scrum Guide," Nov. 2020. [Online]. Available: https://scrumguides.org/scrum-guide.html. [Accessed: Oct. 25, 2026].